

代码生成 Agent 的 AI Flow

工具 · 架构 · 核心概念 (MCP / Skills / Hooks)

8PT GRID · 2X SYSTEM

从补全到 Agent：三段演进

■ EVOLUTION · 2021 → 2026

代码生成的形态五年里走了三段，本质区别是：[谁在闭环里做决策](#)

2021 —

Autocomplete · 补全

模型预测接下来几行代码，人写主体、接受或拒绝建议。

驾驶位

人

2023 —

Chat · 对话生成

把上下文贴进对话框，生成整个函数；人复制回来改。

驾驶位

人主导 · 模型辅助

2024 —

Agent · 自主执行

模型自己读仓库、跑命令、看结果、修正，循环到完成。

驾驶位

模型执行 · 人审查

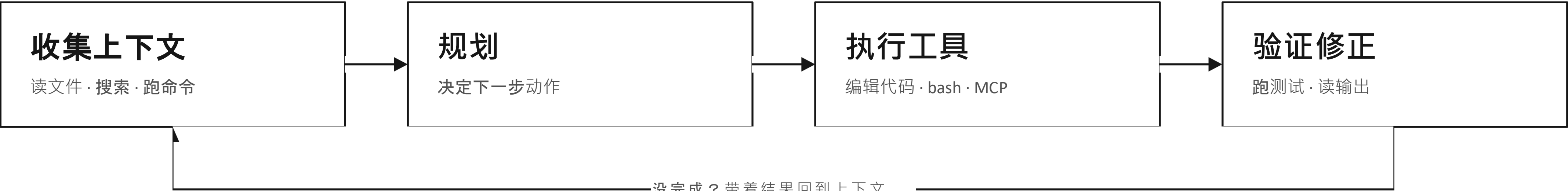
分水岭不是「一次生成多少」，而是[模型自己是否在「执行 → 验证 → 修正」的循环里](#)



架构总览：一个循环，四个组件

剥掉产品差异，所有 coding agent 的骨架相同：一个循环、四个组件

LOOP · 循环到任务完成



COMPONENT 01

LLM · 推理核心

决定下一步、生成代码与命令

COMPONENT 02

上下文管理

文件、搜索、记忆——模型看到什么是能力上限

COMPONENT 03

工具执行

bash、编辑器、MCP 工具——agent 的手脚

COMPONENT 04

权限系统

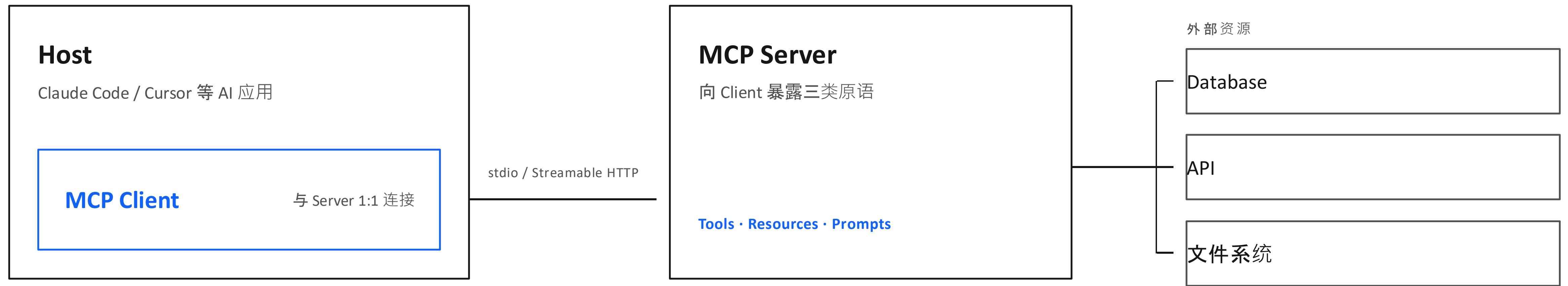
人审或自动放行——安全的边界

循环是灵魂：模型不是一次性回答，而是行动、看结果、再决定下一步



Model Context Protocol, Anthropic 2024 年开源的协议，让任何 AI 应用以统一方式接入外部工具与数据——「AI 应用的 USB-C」

ARCHITECTURE · HOST / CLIENT / SERVER



PRIMITIVE 01

Tools
可调用的动作：查库、发请求

PRIMITIVE 02

Resources
可读取的数据：文件、记录

PRIMITIVE 03

Prompts
预设任务模板

Skills：可复用的程序性知识包

Skills 把「这类任务怎么做」打包成文件夹：平时只有一句话驻留上下文，命中才加载全文



STEP 01 · 常驻

几十 token 的「名片」

上下文里只有 name + description，用于判断是否命中

STEP 02 · 命中

加载 SKILL.md 全文

任务与 description 匹配后，分步指令才进入上下文

STEP 03 · 展开

执行中按需读附件

scripts / references 用到哪个读哪个，上下文始终精简



Hooks：生命周期上的确定性护栏

Hooks 在生命周期事件上挂确定性的 shell 命令——[护栏不靠模型自觉](#)

PreToolUse	工具调用前	退出码 2 阻断调用：禁 rm -rf、禁改 .env
PostToolUse	工具调用后	自动跑 prettier、eslint --fix，结果回喂模型
UserPromptSubmit	用户提交后	注入团队规范、校验敏感信息
Stop	agent 停止前	检查测试是否通过，未过则强制继续

另有 SessionStart / SessionEnd / SubagentStop / PreCompact 等事件，覆盖会话与子 agent 全生命周期



工具全景：同一骨架，不同入口

主流 coding agent 形态各异，但骨架都是同一个循环；差别在入口与上下文工程



Claude Code

Terminal CLI

终端里的 agent：读仓库、跑命令、直接提 PR



Cursor

AI IDE

编辑器原生 agent 模式，codebase 检索与内联编辑



GitHub Copilot

IDE Plugin

从补全先驱到 agent 模式，与 GitHub 仓库深度绑定



Windsurf

AI IDE

Cascade 流式 agent，强调长任务的连续性



OpenAI Codex

Cloud + CLI

云端沙箱跑任务，支持多任务并行与 CLI 本地调用



Cline

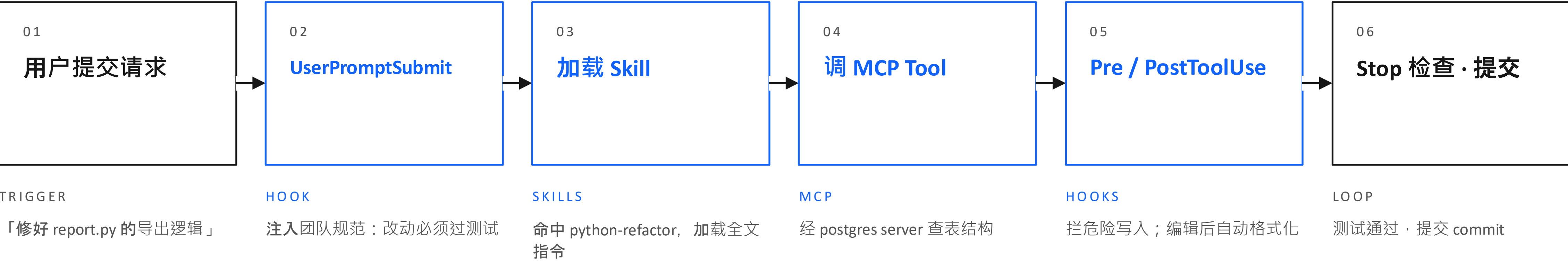
VS Code Extension

开源 agent 扩展，模型与权限完全自托管



一次任务：三个概念在哪出现

把三个概念串进一个真实任务，看它们各自出现在循环的哪一步



蓝框 = 扩展点：Hooks 守门、Skills 供知识、MCP 供连接，黑框 = 循环本身的进出

总结：连接、知识、护栏

LAYER 01

MCP Server

连接层——把外部工具与数据，以统一协议接进任何 AI 应用

LAYER 02

Skills

知识层——把任务方法与团队规范，变成常驻可命中的能力包

LAYER 03

Hooks

护栏层——在生命周期上挂确定性拦截与审计，不靠模型自觉

连接用 MCP，方法论沉淀用 Skills，红线交给 Hooks——**循环本身，交给模型**

